TEMA 2. Encapsulación

1. En Programación Orientada a Objetos (POO), ¿Qué buscan la **encapsulación** y **la ocultación** de información? Enumera brevemente algunas ventajas de la ocultación de información.

La **encapsulación** es un principio fundamental de la POO que consiste en agrupar datos (atributos) y comportamientos (métodos) relacionados dentro de una única unidad llamada clase.
La **ocultación de información** es un concepto complementario que busca restringir el acceso directo a los datos internos de un objeto, permitiendo que solo puedan ser manipulados a través de métodos públicos controlados.

El objetivo principal de la ocultación es proteger la integridad de los datos y facilitar el mantenimiento del código. Al ocultar los detalles de implementación, se crea una barrera entre el "qué hace" un objeto (su interfaz) y el "cómo lo hace" (su implementación interna).

Entre las ventajas de la ocultación de información se encuentran:

> - (1) **Garantizar** que el estado interno es siempre válido.
> - (2) **Evitar** que otro código pueda acceder a partes de mi código que no quiero.
> - (3) **Facilitar** poder cambiar partes sin afectar a otros.

2. ¿Qué se entiende por la **interfaz pública** de un objeto o clase en POO? Describe brevemente cómo se relaciona con la ocultación de información.

La **interfaz pública** de una clase es el conjunto de métodos y atributos (los miembros) que están accesibles desde fuera de la clase, es decir, desde otras partes del programa.

3. Brevemente: ¿Por qué hay que ser conscientes y diseñar con cuidado la **interfaz pública** de una clase? ¿Es fácil cambiarla?

Cambiar la interfaz pública no es fácil, si se cambia tiene más consecuencias que si se cambian partes ocultas, ya que otros códigos pueden depender de esa interfaz.

4. ¿Qué son las **invariantes de clase** y por qué la ocultación de información nos ayuda?

Las **invariantes de clase** son condiciones o propiedades que deben cumplirse siempre para los atributos de un objeto, garantizando que el objeto se mantenga en un estado válido y coherente. Son restricciones lógicas que definen qué estados son válidos para un objeto. Por ejemplo, que la edad de una persona no sea negativa.

La ocultación de información es importante porque, al hacer privados los atributos, se obliga a que cualquier modificación pase por métodos públicos controlados, garantizando que desde el exterior no se modifique a valores fuera de las invariantes de clase.

5. Pon un ejemplo de una clase `Punto` en `Java`, con dos coordenadas, `x` e `y`, de tipo `double`, con un método `calcularDistanciaAOrigen`, y que haga uso de la ocultación de información.

```java
public class Punto {
    // Atributos privados (ocultos)
    private double x;
    private double y;

    // Constructor público
    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    // Método público para calcular distancia al origen
    public double calcularDistanciaAOrigen() {
        return Math.sqrt(x * x + y * y);
    }

    // Métodos públicos para acceder a las coordenadas
    public double getX() { return x; }
    public double getY() { return y; }
}
```

¿Cuál es la interfaz pública de la clase `Punto` ?

La **interfaz pública** de la clase `Punto` está formada por el constructor `Punto(double x, double y)` y los métodos `calcularDistanciaAOrigen()` , `getX()` y `getY()` . Estos son los elementos marcados con `public` y representan todo lo que el código exterior puede usar para interactuar con objetos de tipo `Punto` .

¿Qué significa `public` y `private` ?

El modificador **`public`** indica que un elemento (atributo, método o constructor) es accesible desde cualquier parte del programa, desde otras clases y paquetes. Es parte de la interfaz visible de la clase.
Por el contrario, **`private`** indica que un elemento solo es accesible dentro de la propia clase, permaneciendo oculto para el resto del código.

6. En Java, ¿A quiénes se pueden aplicar los modificadores `public` o `private` ?

Dentro de una clase se puede marcar como `public` o `private` casi cualquier miembro: **atributos**, **métodos**, **constructores** y **clases internas**. La elección depende de si se quiere que ese elemento forme parte de la interfaz pública (accesible desde fuera) o se mantenga como detalle de implementación privado.

7. En POO, la visibilidad puede ser pública o privada, pero ¿existen más tipos de visibilidad? ¿Qué ocurre en Java? ¿Y en otros lenguajes?

Sí, existen más niveles de visibilidad además de pública y privada.
En Java hay cuatro niveles:

1. **`private`** (accesible solo dentro de la misma clase)
2. **visibilidad de paquete** o por defecto (sin modificador, accesible desde cualquier clase del mismo paquete)
3. **`protected`** (accesible desde la misma clase, clases del mismo paquete, y subclases incluso si están en otros paquetes)
4. **`public`** (accesible desde cualquier parte del programa).

Esta graduación permite un control más fino sobre qué partes del código pueden acceder a cada elemento.

Otros lenguajes orientados a objetos tienen sistemas similares pero con variaciones. Por ejemplo, C++ tiene `private`, `protected` y `public`, pero no tiene el concepto de visibilidad de paquete. Python es más flexible y usa convenciones: un guion bajo al inicio del nombre (como `_atributo`) sugiere que es privado por convención, aunque técnicamente sigue siendo accesible. C# tiene `private`, `protected`, `internal` (similar a la visibilidad de paquete, pero a nivel de ensamblado), `public`, y combinaciones como `protected internal`. La filosofía varía: algunos lenguajes son más estrictos en el control de acceso, mientras que otros confían más en las convenciones y la responsabilidad del programador.

8. Responde: Los miembros de instancia privados de un objeto están ocultos para (a) otras clases o (b) otras instancias, aunque sean de la misma clase. Pon un ejemplo añadiendo un método `calcularDistanciaAPunto(Punto otro)` y explica la respuesta.

```java
public class Punto {
    private double x;
    private double y;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double calcularDistanciaAOrigen() {
        return Math.sqrt(x * x + y * y);
    }

    // Nuevo método que accede a los atributos privados de otro objeto Punto
    public double calcularDistanciaAPunto(Punto otro) {
        double deltaX = this.x - otro.x;  // Acceso a otro.x (privado)
        double deltaY = this.y - otro.y;  // Acceso a otro.y (privado)
        return Math.sqrt(deltaX * deltaX + deltaY * deltaY);
    }
}
```

La respuesta correcta es **(a) otras clases**. Los miembros privados están ocultos para otras clases, pero **no** para otras instancias de la misma clase.
Como se observa en el ejemplo, el método `calcularDistanciaAPunto` puede acceder directamente a

`otro.x` y `otro.y`, aunque `otro` sea un objeto diferente y esos atributos sean privados. Esto es posible porque el código está dentro de la clase `Punto`, y el modificador `private` otorga acceso a nivel de clase, no a nivel de instancia individual, en otras palabras, es para protegernos entre programadores, no entre instancias.

9. ¿Qué son los métodos "getter" y "setter" en los lenguajes orientados a objetos?

- Un **getter** es un método que devuelve el valor de un atributo privado, permitiendo leer su contenido sin acceder directamente al atributo.
- Un **setter** es un método que permite modificar el valor de un atributo privado, recibiendo el nuevo valor como parámetro.

En Java, la convención de nombres es usar `get` o `set` seguido del nombre del atributo con la primera letra en mayúscula, como `getX()` o `setNombre(String nombre)`.

Los getters y setters permiten incluir lógica adicional: validación de datos en los setters (por ejemplo, verificar que un valor no sea negativo), cálculos o transformaciones en los getters (como redondear un valor antes de devolverlo), o incluso cambiar la implementación interna sin afectar al código que usa la clase.

10. Cuando nos referimos a que la ocultación de información mejora la "seguridad" del programa, ¿nos referimos a que no pueda ser "hackeado"?

No, se refiere a la **seguridad o robustez del código**. Es una seguridad en el sentido de que el programa esté protegido contra errores de programación, usos incorrectos accidentales, y estados inconsistentes.

La ocultación de información mejora esta seguridad porque previene que el código externo modifique los datos de formas no previstas que podrían romper las invariantes de clase o causar comportamientos incorrectos.

La encapsulación contribuye a la seguridad general del software al reducir la superficie de posibles errores y hacer el código más predecible y controlable. Un programa bien encapsulado es más resistente a errores, tanto accidentales como intencionales por parte de otros desarrolladores que usen nuestro código.

11. ¿Qué diferencia hay entre **miembro de instancia** y **miembro de clase**?

- Los **miembros de instancia** son atributos o métodos que pertenecen a cada objeto individual creado a partir de una clase. Cada instancia tiene su propia copia de estos atributos con valores potencialmente diferentes.
- Los **miembros de clase** (también llamados miembros estáticos) pertenecen a la clase misma, no a las instancias individuales. Existe una única copia compartida por todos los objetos de esa clase.

¿Los miembros de clase también se pueden ocultar?

Sí, los miembros de clase también se pueden ocultar usando los mismos modificadores de acceso (`private` , `public` , etc.).

12. Brevemente: ¿Tiene sentido que los constructores sean privados?

Sí, tiene sentido en ciertos casos específicos:

- Controlar el número de instancias que se crean.
- La clase solo tiene miembros de clase (`Math`).
- Quiero crear objetos solo a través de "métodos factory".
- Singleton.

En definitiva, los constructores privados son una herramienta de encapsulación que permite controlar completamente cómo se crean las instancias de una clase, delegando esa responsabilidad a métodos estáticos que pueden aplicar reglas o restricciones específicas.

13. ¿Cómo se indican los **miembros de clase** en Java?

En Java, los miembros de clase se indican con la palabra clave **`static`** .

Pon un ejemplo, en la clase `Punto` definida anteriormente, para que incluya miembros de clase que permitan saber cuáles son los valores `x` e `y` máximos que se han

establecido en todos los puntos que se hayan creado hasta el momento.

```
public class Punto {
    // Atributos de instancia (privados)
    private double x;
    private double y;

    // Atributos de clase (estáticos) para rastrear máximos
    private static double maxX = Double.NEGATIVE_INFINITY;
    private static double maxY = Double.NEGATIVE_INFINITY;

    public Punto(double x, double y) {
        this.x = x;
        this.y = y;

        // Actualizar los máximos si es necesario
        if (x > maxX) {
            maxX = x;
        }
        if (y > maxY) {
            maxY = y;
        }
    }

    // Métodos de clase (estáticos) para acceder a los máximos
    public static double getMaxX() {return maxX;}
    public static double getMaxY() {return maxY;}
}
```

14. Como sería un método factoría dentro de la clase `Punto` para construir un `Punto` a partir de dos coordenadas, pero que las redondee al entero más cercano. Escribe sólo el código del método, no toda la clase ¿Has usado `static` ?

```
public static Punto newPuntoRedondeado(double x, double y) {
    double xRedondeada = Math.round(x);
    double yRedondeada = Math.round(y);
    return new Punto(xRedondeada, yRedondeada);
}
```

Sí, se ha usado `static`. A veces hasta se ocultan los constructores para forzar que se usen métodos factoría.

15. Cambia la implementación de `Punto`. En vez de dos `double`, emplea un array interno de dos

posiciones, intentando no modificar la interfaz pública de la clase.

```java
public class Punto {
    // Cambio en la implementación: ahora usa un array
    private double[] coordenadas;

    private static double maxX = Double.NEGATIVE_INFINITY;
    private static double maxY = Double.NEGATIVE_INFINITY;

    // La interfaz pública del constructor permanece igual
    public Punto(double x, double y) {
        coordenadas = new double[2];
        coordenadas[0] = x;
        coordenadas[1] = y;

        if (x > maxX) maxX = x;
        if (y > maxY) maxY = y;
    }

    // Los métodos públicos mantienen la misma firma
    public double calcularDistanciaAOrigen() {
        return Math.sqrt(coordenadas[0] * coordenadas[0] +
                         coordenadas[1] * coordenadas[1]);
    }

    public double getX() {return coordenadas[0];}
    public double getY() {return coordenadas[1];}

    public double calcularDistanciaAPunto(Punto otro) {
        double deltaX = this.coordenadas[0] - otro.coordenadas[0];
        double deltaY = this.coordenadas[1] - otro.coordenadas[1];
        return Math.sqrt(deltaX * deltaX + deltaY * deltaY);
    }

    public static double getMaxX() {return maxX;}
    public static double getMaxY() {return maxY;}

    public static Punto newPuntoRedondeado(double x, double y) {
        return new Punto(Math.round(x), Math.round(y));
    }
}
```

Este ejemplo demuestra perfectamente el valor de la encapsulación.
Al haber mantenido privados los atributos `x` e `y` originales y haber proporcionado acceso solo a través de métodos públicos, se ha podido cambiar completamente la representación interna (de dos variables separadas a un array) sin modificar la interfaz pública.

Todo el código que usaba la clase `Punto` seguirá funcionando exactamente igual, sin necesidad de ningún cambio.

16. Si un atributo va a tener un método "getter" y "setter" públicos, ¿no es mejor declararlo público?

No, no es mejor declararlo público. Usar getters y setters en lugar de atributos públicos ofrece varias ventajas importantes. También permiten añadir lógica adicional en el futuro.

¿Cuál es la convención más habitual sobre los atributos, que sean públicos o privados?

La **convención más habitual** en programación orientada a objetos es que **los atributos sean privados**. Solo se proporcionan getters y setters públicos cuando realmente se necesita acceso externo a esos valores.

¿Tiene esto algo que ver con las "invariantes de clase"?

Un setter puede validar que el nuevo valor no viole las invariantes antes de asignarlo.

Con un atributo público, es imposible prevenir asignaciones inválidas. Además, en el futuro se podría necesitar añadir validación o lógica adicional, y tener getters/setters desde el principio lo permite sin cambiar la interfaz pública.

17. ¿Qué significa que una clase sea **inmutable**?

Una clase es **inmutable** cuando sus objetos no pueden cambiar de estado después de ser creados.

¿Qué es un método modificador? ¿Un método modificador es siempre un "setter"?

Un **método modificador** es cualquier método que cambia el estado interno de un objeto, es decir, que modifica el valor de uno o más atributos. No, un método modificador no es siempre un setter. Los setters son el tipo más directo de método modificador. En una clase inmutable, no existen métodos modificadores.

¿Tiene ventajas que una clase sea inmutable?

Sí, la inmutabilidad tiene ventajas significativas:

> - Son **seguros en entornos concurrentes**.
> - Son más **simples de entender y usar** (su estado nunca cambia, eliminando efectos secundarios inesperados).
> - Facilitan el **razonamiento sobre el código** (si se pasa un objeto inmutable a un método, se sabe que no será modificado).
> - Su comportamiento es más predecible y **reducen la posibilidad de errores** relacionados con cambios de estado inadvertidos.

La desventaja es que operaciones que implican "modificar" un objeto requieren crear uno nuevo, lo que puede ser menos eficiente en ciertos casos.

18. ¿Es recomendable incluir métodos "setter" siempre y como convención?

No, no es recomendable incluir setters automáticamente para todos los atributos como convención. Los setters hacen la clase mutable.

19. ¿La clase `String` en Java es mutable o inmutable?

La clase `String` en Java es **inmutable**. Una vez creado un objeto `String`, su contenido no puede modificarse.

Todos los métodos que parecen "modificar" una cadena (como `toUpperCase()`, `substring()`, `replace()`, etc.) en realidad devuelven un **nuevo objeto** `String` con el resultado, dejando el original sin cambios. Esta inmutabilidad proporciona las ventajas mencionadas anteriormente: seguridad en concurrencia, posibilidad de compartir cadenas sin riesgo, y comportamiento predecible.

¿Qué ocurre al concatenar dos cadenas? ¿Qué debemos hacer si vamos a hacer una operación que implique concatenar muchas veces para construir paso a paso una cadena muy larga?

Al concatenar dos cadenas con el operador `+` o el método `concat()`, se crea un **nuevo objeto** `String` que contiene el resultado de la concatenación. Las cadenas originales permanecen

inalteradas. Por ejemplo, en `String s1 = "Hola"; String s2 = s1 + " Mundo";` , se crea un nuevo objeto para `s2` , y `s1` sigue conteniendo solo "Hola". Esto significa que cada concatenación implica crear un nuevo objeto y copiar todo el contenido, lo cual es ineficiente cuando se hacen muchas concatenaciones.

Si se necesita concatenar repetidamente para construir una cadena larga, se debe usar la clase **`StringBuilder`** (o `StringBuffer` si se requiere sincronización para múltiples hilos). `StringBuilder` es una clase **mutable** diseñada específicamente para construir cadenas de forma eficiente. Permite añadir texto con el método `append()` sin crear nuevos objetos en cada operación, modificando un buffer interno. Al finalizar, se llama a `toString()` para obtener el `String` inmutable resultante. Por ejemplo:

```
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append("texto ").append(i).append(" ");
}
String resultado = sb.toString();
```

Este enfoque es mucho más eficiente que concatenar con `+` en un bucle, evitando la creación de miles de objetos temporales.

20. En POO ¿Cómo se comparan objetos de una misma clase? ¿Por su contenido o por su identidad?

En POO, los objetos se pueden comparar de dos formas:

- Por **identidad** (si son el mismo objeto en memoria), en java operador `==` .
- Por **contenido** (si tienen los mismos valores en sus atributos, aunque sean objetos diferentes). Se usa el método `equals()` .

¿Qué es el método equals en Java? ¿Qué hace por defecto?

El método **`equals()`** es un método definido en la clase `Object` . **Por defecto**, si no se sobrescribe, `equals()` se comporta igual que `==` . Si se quiere comparar objetos por su contenido, es necesario sobrescribir el método `equals()` en la clase para que compare los atributos relevantes.

¿Cómo se deben comparar dos cadenas en Java?

Para comparar dos cadenas en Java, se debe usar **`equals()`** , nunca `==` .

```
String s1 = new String("Hola");
String s2 = new String("Hola");
System.out.println(s1 == s2);        // false (diferentes objetos en memoria)
System.out.println(s1.equals(s2));   // true (mismo contenido)
```

21. ¿Qué son las clases "wrapper" en un lenguaje de programación orientado a objetos?

Las clases **"wrapper"** (envoltorio) son clases que encapsulan tipos de datos primitivos para tratarlos como objetos. Java tiene tipos primitivos. Para cada tipo primitivo, Java proporciona una clase wrapper:

> - `Integer` para `int` ,
> - `Double` para `double` ,
> - `Boolean` para `boolean` ,
> - `Character` para `char` , etc.

¿Cómo se hace? ¿Es un proceso automático?

En Java, el proceso de convertir entre primitivos y wrappers puede ser manual o automático. Desde Java 5 se puede:

```
Integer i = 7; // Autoboxing
Integer j = Integer.valueOf(7); // Manual
int x = i; // Unboxing automático
int y = i.intValue(); // Unboxing manual
```

❗ Hay que tener cuidado con el unboxing de referencias `null` , que causaría una excepción.

¿Qué ventajas tienen?

- Métodos útiles, como Integer.parseInt("string");
- Pueden usarse en contextos donde se necesitan objetos, como en colecciones genéricas (`List<Integer>`).

¿Todos los lenguajes orientados a objetos tienen tipos primitivos y necesitan wrappers?

No todos los lenguajes orientados a objetos tienen tipos primitivos. Por ejemplo, en lenguajes como Python todo es un objeto, incluyendo los números, por lo que no necesitan clases wrapper. Java y C# sí tienen primitivos por eficiencia y por tanto necesitan wrappers.

22. ¿En POO qué es un **tipo de dato enumerado**?

Un **tipo de dato enumerado** es un tipo que define un conjunto fijo y limitado de valores constantes con nombre.

¿En Java, un tipo de dato enumerado es una clase?

En Java, un enumerado es una clase, pero las instancias que tiene son finitas y conocidas de antemano.
Al ser una clase, se le pueden crear atributos y métodos, aunque con constructores privados.

¿Qué ventajas tienen en términos de encapsulación los enumerados en Java?

Al ser clases completas, pueden encapsular lógica relacionada con cada valor: en vez de tener constantes dispersas y largos `switch` o condicionales, cada valor del enumerado puede tener su propio comportamiento. Por ejemplo, un enumerado `Operacion` con valores `SUMA`, `RESTA`, etc., podría tener un método abstracto `calcular(double a, double b)` que cada valor implementa de forma diferente, encapsulando la lógica de cada operación dentro del valor correspondiente.

23. Crea un tipo enumerado en Java que se llame `Mes`, con doce posibles instancias y que además proporcione métodos para obtener cuántos días tiene ese mes, el ordinal de ese mes en el año (1-12), empleando atributos privados y constructores del tipo enumerado. Añade además cuatro métodos para devolver si ese mes tiene algunos días de invierno, primavera, verano u otoño, indicando con un booleano el hemisferio (norte o sur, parámetro `enHemisferioNorte`). Es decir: `esDePrimavera(boolean esHemisferioNorte)`, `esDeVerano(boolean esHemisferioNorte)`,

esDeOtoño(boolean esHemisferioNorte) , esDeInvierno(boolean esHemisferioNorte)

```
public enum Mes {
    // Instancias del enumerado con sus valores
    ENERO(31, 1),
    FEBRERO(28, 2),
    MARZO(31, 3),
    ABRIL(30, 4),
    MAYO(31, 5),
    JUNIO(30, 6),
    JULIO(31, 7),
    AGOSTO(31, 8),
    SEPTIEMBRE(30, 9),
    OCTUBRE(31, 10),
    NOVIEMBRE(30, 11),
    DICIEMBRE(31, 12);

    // Atributos privados
    private final int dias;
    private final int ordinal;

    // Constructor privado (implícito en enums)
    Mes(int dias, int ordinal) {
        this.dias = dias;
        this.ordinal = ordinal;
    }

    // Método para obtener los días del mes
    public int getDias() {
        return dias;
    }

    // Método para obtener el ordinal del mes (1-12)
    public int getOrdinal() {
        return ordinal;
    }

    // Método para determinar si es un mes de primavera
    public boolean esDePrimavera(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            return ordinal >= 3 && ordinal <= 5; // marzo, abril, mayo
        } else {
            return ordinal >= 9 && ordinal <= 11; // septiembre, octubre, noviembre
        }
```

```
    }

    // Método para determinar si es un mes de verano
    public boolean esDeVerano(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            return ordinal >= 6 && ordinal <= 8; // junio, julio, agosto
        } else {
            return ordinal == 12 || ordinal <= 2; // diciembre, enero, febrero
        }
    }

    // Método para determinar si es un mes de otoño
    public boolean esDeOtoño(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            return ordinal >= 9 && ordinal <= 11; // septiembre, octubre, noviembre
        } else {
            return ordinal >= 3 && ordinal <= 5; // marzo, abril, mayo
        }
    }

    // Método para determinar si es un mes de invierno
    public boolean esDeInvierno(boolean enHemisferioNorte) {
        if (enHemisferioNorte) {
            return ordinal == 12 || ordinal <= 2; // diciembre, enero, febrero
        } else {
            return ordinal >= 6 && ordinal <= 8; // junio, julio, agosto
        }
    }
}
```

Este enumerado demuestra varias características importantes de la encapsulación en Java. Los atributos `dias` y `ordinal` son privados y finales (inmutables), garantizando que no puedan ser modificados después de la construcción. El constructor es privado (aunque en los enums esto es implícito y no se puede hacer público), asegurando que solo existan las doce instancias declaradas.

Cada instancia del enumerado (ENERO, FEBRERO, etc.) tiene sus propios valores para días y ordinal, encapsulados dentro de ella. Los métodos públicos proporcionan acceso controlado a esta información y permiten realizar operaciones relacionadas con los meses, como determinar la estación según el hemisferio. El código que usa este enum puede invocar, por ejemplo, `Mes.JULIO.esDeVerano(true)` que devolverá `true` para el hemisferio norte. La encapsulación garantiza que es imposible crear un "mes inválido" o modificar las propiedades de los meses existentes.